Go Concurrency
Interview Practice Questions

Goroutines · Channels · Synchronization · Context

Each question is written the way an interviewer would state it. Read the prompt and constraints carefully before writing code. Try to solve each problem without looking up the answer, then run your solution with `-race` and verify it terminates cleanly.

Section 1 — Coordination & Alternation

Q1. Print Even and Odd Numbers Alternately

Write a Go program using two goroutines: one prints only even numbers, the other prints only odd numbers. The final output should be the numbers from 1 to N printed in strict ascending order (1, 2, 3, 4, ...).

Constraints:

- Use two goroutines, one for evens and one for odds.
- The main goroutine must wait for both to finish before exiting.
- Do not use `time.Sleep` for synchronization.
- N should be configurable (e.g., N = 20).

Follow-ups the interviewer may ask:

- What happens if you use a buffered channel of size 1 instead of unbuffered?
- How would you extend this to K goroutines printing in round-robin order?
- Can you do this using only `sync.Mutex` and `sync.Cond` instead of channels?

Q2. Three Goroutines Printing A, B, C in Sequence

Spawn three goroutines. The first prints 'A', the second prints 'B', the third prints 'C'. They must coordinate so that the combined output is 'ABCABCABC...' repeated exactly N times.

Constraints:

- Each goroutine may only print its assigned letter.
- No goroutine may print out of turn.
- The program must terminate cleanly after N cycles.

Follow-ups the interviewer may ask:

- Generalize to K goroutines printing letters in order.
- What if you wanted the order to be configurable at runtime?

Q3. Ping-Pong Between Two Goroutines

Implement a ping-pong game between two goroutines. Goroutine 1 prints 'ping', then goroutine 2 prints 'pong'. They alternate exactly N times and then both exit.

Constraints:

- Use channels for coordination.
- Both goroutines must exit cleanly — no leaks.
- Main must wait for both before returning.

Follow-ups the interviewer may ask:

- How would you add a timeout so the game ends if either side takes too long?
- Can you implement this with a single shared channel? With two channels? Which is cleaner and why?

Q4. Print 1 to N Using K Goroutines in Order

Given N and K, spawn K goroutines that together print the numbers 1, 2, 3, ..., N in strict order. Goroutine 0 prints 1, goroutine 1 prints 2, ..., goroutine (K-1) prints K, then goroutine 0 prints K+1, and so on (round-robin).

Constraints:

- Exactly K goroutines, no more.
- Output must be in strict ascending order.
- Each goroutine prints only the numbers assigned to it (by index mod K).

Follow-ups the interviewer may ask:

- What is the bottleneck as K grows? Does throughput improve?
- How would you detect a goroutine leak in this code?

Q5. Concurrent FizzBuzz with Four Goroutines

Implement FizzBuzz using four goroutines: one prints 'Fizz' (multiples of 3, not 5), one prints 'Buzz' (multiples of 5, not 3), one prints 'FizzBuzz' (multiples of 15), and one prints numbers (everything else). The combined output, for n = 1 to N, must exactly match standard FizzBuzz.

Constraints:

- Exactly four goroutines.
- Each goroutine handles only its assigned category.
- The output must be deterministic and correct.

Follow-ups the interviewer may ask:

- How do you avoid a goroutine spinning while waiting for its turn?
- What if N is extremely large — would your solution scale?

Section 2 — Producer-Consumer Patterns

Q6. Bounded Buffer with Single Producer and Single Consumer

Implement a producer-consumer system. The producer generates integers 1 through N and sends them on a channel. The consumer reads them and prints their squares. The buffer capacity must be exactly 5.

Constraints:

- Use a buffered channel of capacity 5.
- Producer must close the channel when done.
- Consumer must exit cleanly when the channel is closed.
- Main waits for the consumer to finish before exiting.

Follow-ups the interviewer may ask:

- What happens if the producer panics mid-way? How do you make this robust?
- What if you used an unbuffered channel — what changes in behavior?

Q7. Multiple Producers, Single Consumer

Spawn P producer goroutines, each generating some integers, all sending to one shared channel. A single consumer reads everything and prints the total sum. The consumer must know when all producers are done.

Constraints:

- P producers, 1 consumer.
- Only the consumer prints output.
- Channel must be closed exactly once, after all producers finish.
- No goroutine leaks.

Follow-ups the interviewer may ask:

- Who should close the channel? Why not a producer?
- How would `sync.WaitGroup` help here?

Q8. Worker Pool

You are given a slice of N jobs (each job is just an integer to be squared). Implement a worker pool of W workers that processes all jobs concurrently and returns the results in a slice. W should be configurable (e.g., 4).

Constraints:

- Exactly W workers process jobs.
- All N results must be collected.
- All goroutines must exit cleanly after completion.
- The order of results does not need to match input order, but explain how you would preserve it if asked.

Follow-ups the interviewer may ask:

- How do you preserve input ordering in the output?
- What if a job can fail — how do you propagate errors?
- How would you add cancellation via `context.Context`?

Q9. Rate-Limited Worker Pool

Extend the worker pool above so that the system processes at most X jobs per second across all workers combined. Use a token-bucket or ticker-based approach.

Constraints:

- Rate is enforced globally, not per worker.
- Workers should block (not busy-wait) when waiting for a token.
- Clean shutdown when all jobs are done.

Follow-ups the interviewer may ask:

- What is the difference between `time.Tick` and `time.NewTicker`?
- Why is `time.Tick` dangerous in long-running programs?

Q10. Three-Stage Pipeline

Build a pipeline of three stages connected by channels: stage 1 generates integers 1..N, stage 2 squares them, stage 3 prints them. Each stage runs in its own goroutine.

Constraints:

- Each stage is a separate goroutine.
- Stages communicate only via channels.
- Pipeline must shut down cleanly when input is exhausted.
- No goroutine leaks.

Follow-ups the interviewer may ask:

- How do you add cancellation so a downstream cancel stops upstream work?
- What changes if stage 2 is slow — where does back-pressure show up?

Section 3 — Fan-Out / Fan-In

Q11. Fan-Out Across N Workers

Given an input channel of jobs, distribute them across N worker goroutines. Each worker processes its job (e.g., compute square) and writes the result to its own output channel. Return all N output channels.

Constraints:

- Exactly N workers.
- Each worker has its own output channel.
- Workers exit cleanly when the input channel is closed.

Follow-ups the interviewer may ask:

- How would you merge those N output channels into a single one?

Q12. Fan-In: Merge Multiple Channels into One

Write a function `Merge(channels ...<-chan int) <-chan int` that takes any number of input channels and returns a single output channel containing all the values from all inputs. The output channel must close only after all input channels have closed.

Constraints:

- Function signature is fixed.
- Output channel must be closed exactly once, after the last input is drained.
- No goroutine leaks.

Follow-ups the interviewer may ask:

- How does this scale with many channels? Any concerns?
- Can you implement this generically using Go generics?

Q13. First Response Wins

You need to query three replicas of a service. Each replica responds after a random delay. Return the first response and cancel the other two so they don't do unnecessary work.

Constraints:

- Use `context.Context` for cancellation.
- Only one result is returned to the caller.
- The other goroutines must exit promptly — no leaks.

Follow-ups the interviewer may ask:

- What if all three fail — how do you report that?
- What if you needed the first two responses instead of just the first?

Section 4 — Synchronization Primitives

Q14. Build a Semaphore Using a Buffered Channel

Implement a counting semaphore that limits concurrent access to a resource to at most K simultaneous holders. Use a buffered channel as the underlying primitive. Expose `Acquire()` and `Release()` methods. Demonstrate it by spawning 20 goroutines that each 'do work' but with at most K = 3 running concurrently.

Constraints:

- Do not use `sync.Mutex` or `sync.RWMutex`.
- Implement using only a channel.
- All goroutines must complete and the program must exit cleanly.

Follow-ups the interviewer may ask:

- Add a `TryAcquire` method that returns false if the semaphore is full.
- Add an `AcquireContext(ctx)` method that respects cancellation.

Q15. Implement sync.Once From Scratch

Without using `sync.Once`, implement your own `Once` type that has a `Do(f func())` method. Calling `Do` many times concurrently must result in `f` being executed exactly once. All other callers must block until that single execution completes.

Constraints:

- Must be safe under concurrent calls.
- Subsequent calls after the first execution must be very cheap (no locking on the fast path if possible).
- Do not use `sync.Once`.

Follow-ups the interviewer may ask:

- Implement it first with `sync.Mutex`, then optimize using `sync/atomic`.
- What is the double-checked locking pattern, and why is it tricky to get right?

Q16. Barrier for N Goroutines

Implement a barrier: N goroutines each do some work, then call `Wait()` on the barrier. None of them proceed past `Wait()` until all N have arrived. Once all arrive, they all unblock simultaneously.

Constraints:

- Must work for any N specified at construction.
- Must be reusable for multiple barrier cycles (extra credit).
- No leaks.

Follow-ups the interviewer may ask:

- How is this different from `sync.WaitGroup`?
- What if one goroutine never arrives — how do you handle that?

Q17. Concurrent-Safe Counter

Implement a counter type with `Inc()`, `Dec()`, and `Value()` methods. Spawn 100 goroutines, each calling `Inc()` 1000 times. Verify the final value is exactly 100,000.

Constraints:

- Provide three implementations and compare them: (1) `sync.Mutex`, (2) `sync/atomic`, (3) a single goroutine owning the state, updated via channels.
- Run with `-race`; verify no races.

Follow-ups the interviewer may ask:

- Benchmark all three and discuss the performance differences.
- When would you actually prefer the channel-based approach despite it being slower?

Section 5 — Channel Patterns & Pitfalls

Q18. Non-Blocking Send and Receive

Given a channel `ch`, write code that attempts to send a value but does not block if the channel is full. Similarly, write code that attempts to receive but does not block if the channel is empty. In both cases, print whether the operation succeeded.

Constraints:

- Must not block under any circumstances.
- Use `select` with a `default` branch.

Follow-ups the interviewer may ask:

- Why is the `default` branch in `select` so important here?
- What is the behavior of `select` with no `default` when no case is ready?

Q19. Receive with a Timeout

Write a function that tries to receive a value from a channel but gives up after 500 milliseconds. Return the value and a boolean indicating whether it arrived in time.

Constraints:

- Use `select` and `time.After` or `time.NewTimer`.
- Do not leak any timers or goroutines.

Follow-ups the interviewer may ask:

- Why is `time.After` in a hot loop a memory problem?
- Rewrite using `time.NewTimer` with proper `Stop` and reset.

Q20. Broadcast to N Subscribers

Implement a broadcaster: a single publisher sends messages, and any number of subscribers each receive every message. Subscribers can subscribe and unsubscribe at any time.

Constraints:

- All subscribers see all messages published after they subscribed.
- A slow subscriber must not block the publisher or other subscribers (define your policy: drop, buffer, or block — and explain).
- Unsubscribe must clean up resources.

Follow-ups the interviewer may ask:

- What is your policy for slow subscribers, and why?
- How would you bound memory usage when one subscriber falls behind?

Q21. Close a Channel Exactly Once

Multiple goroutines may decide to close a shared channel, but closing a channel twice causes a panic. Design a safe close mechanism so that the channel is closed exactly once no matter how many goroutines try.

Constraints:

- No panics, ever.
- All concurrent `SafeClose` calls return without error after the first.

Follow-ups the interviewer may ask:

- Compare implementations using `sync.Once`, `sync.Mutex`, and `sync/atomic`.
- Why is sending to a closed channel still a panic, and how do you protect against it?

Q22. Goroutine Leak Diagnosis

You are given this code (write it on the board): a function spawns a goroutine that sends a result on a channel. The caller uses `select` with a timeout to receive the result, returning early if the timeout fires. Identify the bug and fix it so no goroutines are leaked even when the timeout fires.

Constraints:

- Fix must not change the function signature.
- Demonstrate the leak before the fix using `runtime.NumGoroutine`.

Follow-ups the interviewer may ask:

- What general lesson does this illustrate about unbuffered channels and timeouts?
- Would using a buffered channel of capacity 1 be a valid fix? Why or why not?

Section 6 — Context & Cancellation

Q23. Cancellable Worker Pool

Take the worker pool from Section 2 and add `context.Context` support. When the context is cancelled, all workers must stop accepting new jobs and exit cleanly. Any in-flight job should be allowed to finish or check the context, your choice (explain your design).

Constraints:

- Pass `ctx` as the first argument to all worker functions.
- Workers must respect cancellation.
- No goroutine leaks after cancellation.

Follow-ups the interviewer may ask:

- What is the difference between `ctx.Done()` closing and `ctx.Err()` returning non-nil?
- Should the worker pool itself return an error when cancelled? What error?

Q24. Graceful Shutdown on SIGINT

Write a program that runs N worker goroutines in an infinite loop, each doing periodic work. On receiving SIGINT (Ctrl+C), the program must stop accepting new work, give workers up to 5 seconds to finish their current iteration, and then exit. If a worker exceeds the deadline, log a warning and exit anyway.

Constraints:

- Use `signal.NotifyContext` or equivalent.
- Use a parent context that's cancelled on SIGINT.
- Use a deadline context for the shutdown grace period.

Follow-ups the interviewer may ask:

- What is the difference between `signal.Notify` and `signal.NotifyContext`?
- How do you ensure the second SIGINT terminates the program immediately?

Q25. Implement WithTimeout Yourself

Without using `context.WithTimeout` or `context.WithDeadline`, build your own version: given a parent context and a duration, return a derived context that is automatically cancelled when the duration elapses or the parent is cancelled, whichever comes first.

Constraints:

- You may use `context.WithCancel` and `time.AfterFunc`.
- Returned `cancel` function must stop the timer to avoid leaks.

Follow-ups the interviewer may ask:

- Why does the standard `WithTimeout` return a `CancelFunc` that callers should always defer?
- What goroutine or memory leak happens if you don't stop the timer?

Section 7 — Real-World Scenarios

Q26. Concurrent Web Crawler

Implement a concurrent web crawler in Go. Given a starting URL and a maximum depth D, fetch the page, extract all links, and recursively crawl them up to depth D. You may mock the fetch function with a fake one that returns hardcoded links.

Constraints:

- Maximum of W concurrent fetches at any time.
- Each URL must be visited at most once.
- Crawler must terminate cleanly once all reachable pages are visited.
- Use `context.Context` so the crawl can be cancelled.

Follow-ups the interviewer may ask:

- How do you make the 'visited' set concurrent-safe? Compare options.
- What if a fetch hangs forever — how do you protect the crawler?
- How do you keep memory bounded if the site is huge?

Q27. Concurrent-Safe Map (Without sync.Map)

Implement a generic concurrent map type with `Get`, `Set`, `Delete`, and `Len` methods. Do not use `sync.Map`. Test it with many goroutines reading and writing concurrently under `-race`.

Constraints:

- All methods must be safe under concurrent use.
- Provide a version using `sync.RWMutex`.
- Optionally provide a sharded version (e.g., 16 shards) and benchmark against the simple version.

Follow-ups the interviewer may ask:

- When would `sync.Map` outperform your implementation?
- What is the typical workload `sync.Map` is optimized for?

Q28. Token-Bucket Rate Limiter

Implement a rate limiter that allows at most R operations per second, with a burst capacity of B. Provide `Allow()` (non-blocking, returns bool) and `Wait(ctx)` (blocking, respects context) methods.

Constraints:

- Tokens refill at rate R.
- Maximum tokens stored is B.
- Must be safe under concurrent use.
- `Wait` must return an error if the context is cancelled first.

Follow-ups the interviewer may ask:

- Compare your implementation to `golang.org/x/time/rate`.
- How would you implement this purely with channels and a ticker?

Q29. Pub-Sub System with Slow Subscriber Handling

Implement a topic-based pub-sub system. Publishers send messages to a topic, and subscribers to that topic each receive a copy. Subscribers can have different speeds. Your system must decide what happens to a slow subscriber.

Constraints:

- Multiple publishers and subscribers per topic.
- Subscribe and unsubscribe operations are concurrent-safe.
- Define and justify your slow-subscriber policy (drop oldest, drop newest, disconnect, etc.).
- No leaks when a subscriber unsubscribes or disconnects.

Follow-ups the interviewer may ask:

- How would you persist messages so a subscriber that reconnects can catch up?
- How would you scale this to millions of subscribers?

Q30. Find and Fix the Deadlock

You are shown the following code (have the interviewer or yourself write a small program that deadlocks: e.g., two goroutines each acquiring two mutexes in opposite order, or a goroutine waiting on an unbuffered channel that nobody sends to). Identify why it deadlocks and fix it.

Constraints:

- Explain the deadlock condition (the four Coffman conditions, or the channel-send/receive mismatch).
- Provide a fix and explain why it works.

Follow-ups the interviewer may ask:

- How does `go run -race` help — or not help — with deadlocks?
- What does Go's runtime do when it detects all goroutines are asleep?

Section 8 — Gotchas Interviewers Love

Q31. Loop Variable Capture

Consider this code (write it out): a `for` loop iterating over a slice, spawning a goroutine in each iteration that prints the loop variable. On Go 1.21 and earlier, this prints unexpected results.

Constraints:

- Explain what gets printed and why.
- Provide at least two ways to fix it (parameter passing, local shadowing).
- Explain what changed in Go 1.22 and why.

Follow-ups the interviewer may ask:

- Does this bug still exist in your version of Go?
- Are there other constructs (e.g., `defer` in a loop) with similar pitfalls?

Q32. WaitGroup Misuse

Show three different ways someone could misuse `sync.WaitGroup`: (1) calling `Add` after `Wait` has started, (2) calling `Done` more times than `Add`, (3) passing the `WaitGroup` by value to a goroutine. For each, explain what goes wrong and how to fix it.

Constraints:

- Demonstrate each bug with a small code example.
- Show the corrected version for each.

Follow-ups the interviewer may ask:

- What does the Go runtime do when `Done` is called too many times?
- When would you choose `errgroup.Group` over `sync.WaitGroup`?

Q33. Select Starvation

Construct a `select` with two cases, where one channel is constantly ready and the other only occasionally. Demonstrate whether the rarely-ready case ever fires. Explain Go's tie-breaking rule for `select`.

Constraints:

- Demonstrate empirically by running the program.
- Reference the language spec on `select` semantics.

Follow-ups the interviewer may ask:

- If both cases are always ready, is the choice deterministic? What does the spec say?
- How could you guarantee fairness if your application required it?

Q34. Nil Channel in Select

Explain what happens when one of the cases in a `select` uses a nil channel. Then show a useful pattern that exploits this behavior to dynamically enable or disable a case.

Constraints:

- Give a runnable example.
- Use the pattern in a realistic scenario (e.g., turning off a producer once a quota is hit).

Follow-ups the interviewer may ask:

- Why does sending to or receiving from a nil channel block forever?
- How is this pattern cleaner than using a boolean flag?

Q35. Map Concurrent Access Panic

Demonstrate the runtime error that Go produces when multiple goroutines write to a plain (non-synchronized) `map`. Then provide three solutions: `sync.Mutex`, `sync.RWMutex`, and `sync.Map`. Discuss when each is appropriate.

Constraints:

- Show the actual panic message.
- Run all solutions under `-race`.

Follow-ups the interviewer may ask:

- Why does Go panic instead of corrupting silently?
- Why is `sync.Map` not always the best default?

Tip: For every question, after you write your solution, run it with `go run -race main.go` and verify the program terminates cleanly. Then ask yourself: what is the worst input? What happens if a goroutine panics? Where could a leak hide?